



USN

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

NMAM INSTITUTE OF TECHNOLOGY, NITTE
Off-Campus Centre of Nitte (Deemed to be University)
Third Semester B.Tech (CBCS) Degree Examinations
December 2025
EC2003-1 – DIGITAL SYSTEM DESIGN WITH VERILOG
(For EC)

Duration: 3 Hours

Max. Marks: 100

Note:

Part – A: Multiple Choice Questions: Answer all **Twenty** questions in the **OMR Sheet** provided. Each question carries equal marks.

Part – B: Descriptive Answer Questions: Answer **Five full** questions choosing **Two full** questions from **Unit – I & Unit – II** each and **One full** question from **Unit – III**.

PART - A: MULTIPLE CHOICE QUESTIONS**20 Marks**

1. Which Verilog code snippet demonstrates the correct use of a User Defined Primitive (UDP) for a 1-bit full adder?

A) `)primitive
full_adder(sum,
carry, A, B, Cin);
output sum, carry;
input A, B, Cin;
table
// A B Cin : sum
carry
0 0 0 : 0 0;
0 1 0 : 1 0;
1 0 0 : 1 0;
1 1 1 : 1 1;
endtable
endprimitive`

B) `primitive
full_adder(sum,
carry, A, B, Cin);
output sum, carry;
input A, B, Cin;
table
// A B Cin : sum
carry
0 0 0 : 0 0;
0 1 0 : 1 0;
1 0 0 : 1 0;
1 1 0 : 0 1;
1 1 1 : 1 1;
endtable
endprimitive`

C) `primitive
full_adder(sum,
carry, A, B, Cin);
output sum, carry;
input A, B, Cin;
table
// A B Cin : sum
carry
0 0 1 : 1 0;
0 1 1 : 0 1;
1 0 1 : 1 1;
1 1 0 : 0 1;
1 1 1 : 1 1;
endtable
endprimitive`

D) Separate
Primitives for Sum
and Carry (Since
UDP can have only
one output)

2. Which one of the following represents a hardware component in Verilog?
 A) Module
 B) Function
 C) Loop
 D) Variable
3. How many 2-to-1 multiplexers are needed to implement a 4-to-1 multiplexer?
 A) 1
 B) 2
 C) 3
 D) 4
4. In Verilog, what is the purpose of the “always” block?
 A) To declare variables
 B) To instantiate modules
 C) To specify simulation time
 D) To describe behavior using procedural statements
5. Which of the following is a valid Verilog code snippet for a 2-to-4 decoder using behavioral modeling?
- A) module
 decoder_2to4(input
 [1:0] in, output reg
 [3:0] out);
 always @(*) begin
 out = 4'b0000;
 case (in)
 2'b00: out[0] =
 1;
 2'b01: out[1] =
 1;
 2'b10: out[2] =
 1;
 2'b11: out[3] =
 1;
 endcase
 end
 endmodule

B) module
 decoder_2to4(input
 [1:0] in, output [3:0]
 out);
 assign out = 1 << in;
 endmodule
- C) module
 decoder_2to4(input
 [1:0] in, output [3:0]
 out);
 and (out[0], ~in[1],
 ~in[0]);
 and (out[1], ~in[1],
 in[0]);
 and (out[2], in[1],
 ~in[0]);
 and (out[3], in[1],
 in[0]);
 endmodule
- D) Behavior model can not be used for modeling a decoder
6. Which of the following is true about Structural modeling in Verilog?
 A) It uses always blocks to describe behavior
 B) It uses assign statements to describe behavior
 C) It connects predefined modules using instantiation
 D) It is used for defining continuous assignments
7. Which of the following circuits is not combinational?
 A) Multiplexer
 B) Decoder
 C) Flip-Flop
 D) Adder

8. In the following Verilog code, what will the output Q be when clk is high and D is 1?
- ```
always@(posedge clk) begin
 Q <= D;
end
```
- A) 0  
B) 1  
C) x  
D) last value of Q
9. To trigger an event, we can use the following operator.
- A) ->  
B) ==>  
C) @  
D) ===
10. @posedge means
- A) Transition from x to 1  
B) Transition from 0 to 1  
C) Transition from z to 1,x  
D) Transition from 1 to 0
11. What is the typical range of counting for a 4-bit asynchronous counter?
- A) 0 to 15  
B) 2 to 15  
C) 1 to 15  
D) 0 to 7
12. Which Verilog statement would you use to shift a 4-bit shift register left by one position?
- A) data <= {data[3:1], data[0]};  
B) data <= {serial\_in, data[3:1]};  
C) data <= data << 1;  
D) data <= {data[3], data[2:0]};
13. How do you model a 3-bit synchronous up-counter in Verilog?
- A) reg [2:0] count; always @(posedge clk) count <= count - 1;  
B) reg [2:0] count; always @(posedge clk) count <= count + 1;  
C) reg [2:0] count; always @\* count <= count + 1;  
D) reg [2:0] count; always @\* count <= count - 1;
14. How does a latch differ from a flip-flop in Verilog?
- A) A latch is edge-triggered, while a flip-flop is level-triggered  
B) A flip-flop is edge-triggered, while a latch is level-triggered  
C) both are same  
D) type of modeling style is different
15. How do you connect the flip-flops in an asynchronous counter?
- A) All flip-flops receive the same clock signal  
B) Each flip-flop's output is connected to the clock of the next flip-flop  
C) Flip-flops are connected in parallel  
D) Flip-flops are connected in series with a common clock
16. In a moore machine, what does the output depend upon?
- A) current state only  
B) Input Only  
C) Previous state & current state  
D) current state and input
17. What is the typical usage of the reg keyword in a RAM module?
- A) To declare input ports  
B) To define output ports  
C) To define storage elements for data  
D) To specify the clock signal
18. What is the primary use of the concept of a "starting state" in a Finite State Machine?
- A) To identify the final state of the machine.  
B) To determine the machine's output.  
C) To define the state where processing begins.  
D) To represent the machine's memory.
19. In a Finite State Machine, what is the role of a transition function?
- A) To determine the initial state.  
B) To determine the final state.  
C) To map a state and an input symbol to a next state.  
D) To provide an output based on the current state.
20. In a moore machine, what does the output depend upon?
- A) current state only  
B) Input Only  
C) Previous state & current state  
D) current state and input

**PART - B: DESCRIPTIVE ANSWER QUESTIONS****Unit – I**

Marks BT\* CO\* PO\*

1. a) Design a Verilog architecture file for a digital component called three\_input\_sum\_logic. This circuit is required to accept three independent single-bit inputs, labeled X, Y, and Z, and must generate two distinct single-bit outputs: a Result signal and a NextCarry signal.

The entire combinational behavior must be defined within a single procedural block where the functional mapping is established exclusively by means of conditional decision statements (if-else) to determine the value of the outputs based on all possible input combinations.

8 L\*3 1 1

- b) Develop a Verilog code for a component named SelectorUnit\_8. This element functions as a dynamic 8:1 Data Concentrator, designed to channel one of eight distinct binary sources to a single output port based on an encoded address signal.

The module must be defined using the dataflow methodology, relying exclusively on a single continuous assignment statement (using the ternary conditional operator) to govern the output behavior.

The port definitions are as follows:

- Input Bus I\_data: An 8-wire vector (I\_data[7:0]) representing the multiple input sources.
- Address Input (A\_sel): A 3-wire control vector (A\_sel) used to specify the active input.
- System Output (Z\_out): A single wire that carries the selected bit value.

The core requirement is to express the entire Z\_out function as a nested structure of conditional operators that evaluates the A\_sel and maps the corresponding bit from I\_data to the output.

8 L3 2 1

2. a) Design a Verilog module called TernarySplitter that acts as a specialized decoder for a single BCD digit. The module must have a 4-bit input named BCD\_Value and two separate 2-bit outputs named MajorPart and MinorPart. The functionality must be based on a finite state mapping that covers the ten valid BCD codes (0 to 9). Specifically, for each valid input in the range 4'b0000 through 4'b1001, the module must output a unique 2-bit MajorPar and a 2-bit MinorPar. These outputs must correspond exactly to the quotient and remainder respectively, when the decimal equivalent of BCD Value is divided by three (3). The implementation must use a procedural always @(\*) block containing a case statement structure to explicitly list and assign the correct output pair for every valid input value. Also write appropriate test bench using task and repeat.

8 L3 1 1

- b) Using Fault Table method, find the minimum number of test vectors required to verify the circuit shown in Figure 2(b).

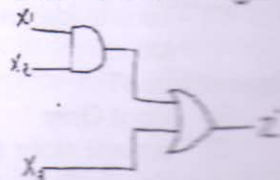


Fig. 2(b)

Using the Fault table method, find the minimum number of test vectors required to verify the circuit shown in Fig. 2(b).

8 L3 2 1

3. a) What are Verilog User-Defined Primitives? Explain with the help of an example program.

8 L3 1 1

b)

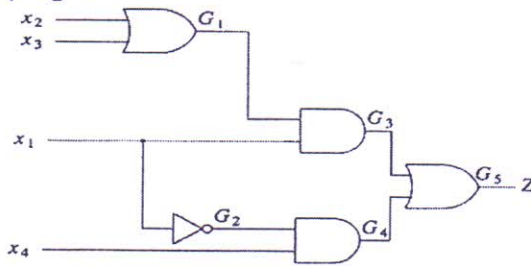


Fig. 3(b)

A logical circuit is shown in Fig. 3(b).

Using Path Sensitization method determine the Test Vector to check:

- Stuck-at-0 fault in the path  $x_2$ -G1-G3-G5-Z
- Stuck-at-1 fault in the path  $x_3$ -G1-G3-G5-Z
- Stuck-at-0 fault in the path  $x_1$ -G2-G4-G5-Z
- Stuck-at-1 fault in the path  $x_4$ -G4-G5-Z

8 L3 2 1

## Unit – II

4. a) Develop a synthesizable Verilog HDL module named DataCache\_Unit that models a Digital Scratchpad Component with an internal storage capacity of 16 locations, each holding 8 bits.

The module must implement synchronous write and asynchronous read access using the following port specifications:

- CLK\_IN (1-bit): Clock signal.
- INDEX\_BUS (4-bits): Address input for memory location selection.
- DATA\_WRITE (8-bits): Data input.
- DATA\_READ\_OUT (8-bits): Data output.
- MODE\_SELECT (1-bit): Control signal.

The logic must satisfy these rules:

1. A data update (write) operation must occur only on the positive edge of CLK\_IN when MODE\_SELECT is High (1). The value on DATA\_WRITE is stored at the location specified by INDEX\_BUS.
2. A data retrieval (read) operation must be performed combinatorially (asynchronously) whenever MODE\_SELECT is Low (0). The data at the location specified by INDEX\_BUS must immediately appear on DATA\_READ\_OUT.

8 L3 3 1

- b) Design a 3-bit counter-like circuit, named Counter3, controlled by the input w. If  $w = 1$ , then the counter increments its contents by 2, wrapping around if the count reaches 6 or 7. Thus, if the present state is 6 or 7, then the next state becomes 0 or 1, respectively. If  $w = 0$ , then the counter subtracts 1 from its contents, acting as a normal down-counter. The module has an asynchronous Reset.

8 L3 3 1

5. a) Develop a Verilog module named `UniOp_Register` that functions as a 4-bit configurable sequential storage element. The design must use a behavioral modeling style within a single synchronous procedural block to manage all state updates.

The component interfaces are:

- `P_Data` (4 bits): The input vector used for parallel data injection.
- `S_Data` (1 bit): The single-wire input used for serial bit ingestion.
- `Control_Code` (2 bits): The mode selection signal.
- `Reg_State` (4 bits): The registered output value.

Implement the logic to enforce the following operational modes, determined by `Control_Code` as shown in Table 5(a):

| Control_Code | Operational Mode    | Technical Action                                                                                                                            |
|--------------|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 2'b00        | Retention Mode      | The <code>Reg_State</code> must hold its current value (No Change).                                                                         |
| 2'b01        | Left Propagation    | The contents of <code>Reg_State</code> shift one position left. The least significant bit (LSB) is filled by the <code>S_Data</code> input. |
| 2'b10        | Right Propagation   | The contents of <code>Reg_State</code> shift one position right. The most significant bit (MSB) is filled by the <code>S_Data</code> input. |
| 2'b11        | Mass Data Overwrite | The <code>Reg_State</code> is immediately loaded with the entire <code>P_Data</code> input vector (Parallel Load).                          |

Table 5(a)

- b) Design a Verilog behavioral model for a 16-bit, binary up/down counter, with asynchronous reset, using a single procedural block. The block diagram for the port definition is shown in Fig.5b. When `Up=1`, the counter will increment. When `Up=0`, the counter will decrement. Also write a suitable testbench using `forever` and `for` loop concept.

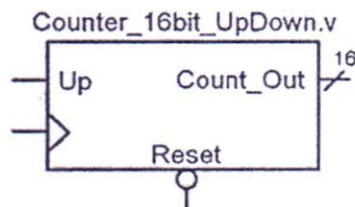


Fig. 5(b)

6. a) Write a Verilog code for Sequence Generator that generates the following sequence 0,1,2,4,9,10,5,6,8,7,0. using behavioural modelling.
- b) Develop a Verilog HDL module named `TimingUnit_Gen` that derives multiple lower-rate periodic signals from a high-speed master reference. The module must accept the following inputs: `Main_Reference` (the fundamental clock input, frequency  $F_{in}$ ) and `Synchronous_Clear` (an active low reset that forces all outputs low, synchronized to the clock). The module must generate two independent output signals: `Slow_Pulse_A` and `Slow_Pulse_B`. `Slow_Pulse_A` must have a frequency of  $F_{in} / 2$ . `Slow_Pulse_B` must have a frequency of  $F_{in} / 20$ . Implement the necessary sequential logic to achieve these frequency divisions and ensure the `Synchronous_Clear` functionality is correctly implemented.

## Unit – III

7. a)

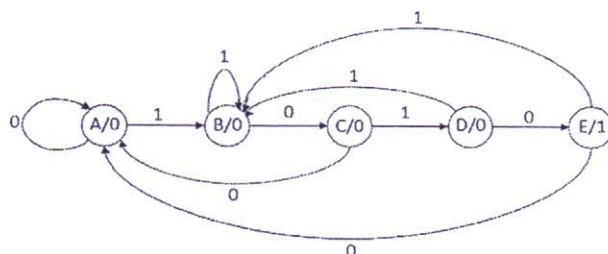


Fig. 7(a)

Explain how the state machine shown in Fig. 7(a) can be used to detect "1010" (non-overlapping). Comment whether the state machine type is Mealy or Moore type.

|    |    |   |   |
|----|----|---|---|
| 6  | L2 | 4 | 1 |
| 10 | L3 | 4 | 1 |

b) Write Verilog code for the state machine shown in Fig. 7(a).

8. a) Create the necessary State Transition Diagram for the overlapping type Bit Pattern Correlator. The diagram must clearly illustrate the minimal set of states required to track the pattern 1001. Since the system must output the result within the same clock cycle as the last bit arrives, use the Mealy model convention. Label all transitions clearly using the format: Seqin/Matchout. Ensure the logic for overlapping detection is correctly reflected in the transitions from the final states.

|   |    |   |   |
|---|----|---|---|
| 6 | L2 | 4 | 1 |
|---|----|---|---|

b) Develop the complete Verilog RTL module for the Bit Pattern Correlator given in Q.8(a). Name the module PatternAnalyzer and use the following port definitions:

- Inputs: Clock, Reset\_sync, Data\_bit.
- Output: Sequence\_valid.

The code must use a modern FSM structure, clearly defining the State Memory, Next-State Logic, and Output Logic in separate blocks. Use a case statement within a combinational block to define the next state transitions, and ensure the Sequence\_valid output is combinatorially assigned (Mealy style).

|    |    |   |   |
|----|----|---|---|
| 10 | L3 | 4 | 1 |
|----|----|---|---|

BT\* Bloom's Taxonomy, L\* Level; CO\* Course Outcome; PO\* Program Outcome

\*\*\*\*\*